

KVServe: Service-Aware KV Cache Compression for Communication-Efficient Disaggregated LLM Serving

Zedong Liu^{1,2,*}, Xinyang Ma^{1,2,*}, Dejun Luo¹, Hairui Zhao², Bing Lu², Wenjing Huang², Yida Gu², Xingchen Liu², Zheng Wei², Jinyang Liu³, Dingwen Tao², Guangming Tan²

¹University of Chinese Academy of Sciences ²Institute of Computing Technology, Chinese Academy of Sciences

³Shanghai Jiao Tong University

ABSTRACT

LLMs are widely adopted in production, pushing inference systems to their limits. Disaggregated LLM serving (e.g., PD separation and KV state disaggregation) improves scalability and cost efficiency, but it also turns KV into an explicit payload crossing network and storage boundaries, making KV a dominant end-to-end bottleneck. Existing KV compression are typically static runtime configurations, despite production service context varies over time in workload mix, bandwidth, and SLO/quality budgets. As a result, a fixed choice can be suboptimal or even increase latency. We present *KVServe*, the first service-aware and adaptive KV communication compression framework for disaggregated LLM serving: *KVServe* (1) unifies KV compression into a modular strategy space with new components and cross-method recomposition; (2) introduces Bayesian Profiling Engine that efficiently searches this space and distills a 3D Pareto candidate set, reducing 50 $\times$ offline search overhead; and (3) deploys a Service-Aware Online Controller that combines an analytical latency model with a lightweight bandit to select profiles under constraints and correct offline-to-online mismatch. Integrated into vLLM and evaluated across datasets, models, GPUs and networks, *KVServe*¹ achieves up to 9.13 $\times$ JCT speedup in PD-separated serving and up to 32.8 $\times$ TTFT reduction in KV-disaggregated serving.

1 INTRODUCTION

Large language models (LLMs) are becoming a general-purpose engine for production inference, yet their autoregressive generation requires maintaining and repeatedly accessing the *Key Value (KV) cache* throughout decoding. In practice, LLM inference is commonly divided into two stages: *prefill* and *decode*. Prefill computes prompt KV cache in parallel and is typically compute-intensive. Decode iteratively generates tokens and reads KV, making it more memory-intensive [50].

To boost throughput and support long contexts at lower cost, production serving systems are moving to *disaggregated* inference architectures. Two representative designs

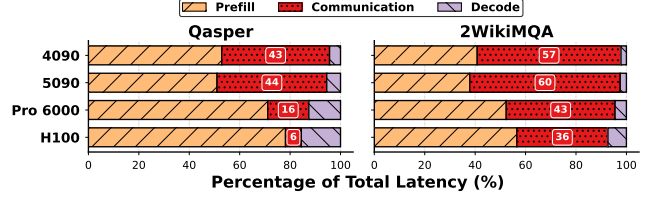


Figure 1: Time breakdown under PD-separated serving.

are *prefill/decode (PD) separation* and *KV state disaggregation* [33, 35, 49]. In PD separation, prefill and decode run on separate GPU nodes to reduce co-location contention and to enable stage-specific scaling. In KV state disaggregation, the KV cache is offloaded to a storage hierarchy or remote KV pool to support longer contexts and cross-request reuse (e.g., RAG, and agents). Unlike monolithic serving where KV is internal GPU state, disaggregation makes KV an explicit payload that must be red across networks [48]. As contexts grow, KV quickly becomes massive (eg. Llama 3.1-70B generates 39.06 GB KV at 128K tokens [36]).

However, this disaggregation introduces a bandwidth-dependent bottleneck: the cost of transferring *KV cache* across network/IO boundaries. Recent agentic and long-context workloads further amplify this pressure: their long inputs and short outputs allow prefill workers to generate KV cache at very high throughput. For example, serving 32K-token requests with Qwen3-235B on a 64-node prefill cluster requires 2.1 Tbps of KV egress bandwidth [34]. In common cloud deployments, cross-cluster bandwidth is often constrained to below 100 Gbps. [1]. Similar limits apply to remote storage/KV pools, where throughput is often below 10 Gbps [26]. This makes KV a dominant cost in disaggregated serving. In our end-to-end experiments (Fig. 1), KV communication time accounts for up to 60% of job completion time. As KV cache grows, this bottleneck will further intensify, calling for optimizations.

Recent work has proposed a range of *KV compression* methods that significantly reduce KV volume with acceptable quality loss. Representative works such as CacheGen [26], KIVI [28], and KVQuant [16] quantize BF16 KV caches to 4-bit or 2-bit and further increase compression ratios via lossless coding. Finer-grained quantization schemes, such as mixed-precision quantization [11, 24, 39], assign different

¹<https://github.com/hpdps-group/KVServe>

*Equal contribution.

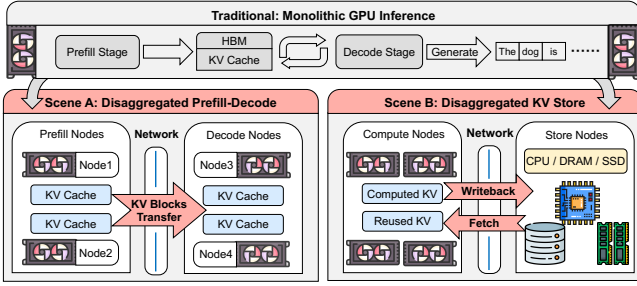


Figure 2: Architecture of disaggregated serving system.

precisions based on layer-level or token-level importance. Other methods improve compressibility and control quality degradation through transforms such as Hadamard [3] or Affine [29] preprocessing.

Despite their effectiveness, these methods are generally *statically configured* at runtime: fixed choice of transforms, quantization granularities, and codecs. A static configuration may reduce latency under some conditions, but can also cause *negative optimization*. This is because the service context in production changes dynamically, including workload type, effective bandwidth, and Service Level Objective (SLO) budgets. Our measurements show that the latency-optimal choice can switch across workloads and bandwidth regimes (detailed in Sec.2.2). In other words, in disaggregated serving, KV compression is not a fixed algorithm choice; it is a *constrained, service-state-dependent strategy selection* problem.

However, achieving service-aware and adaptive KV compression in disaggregated inference is non-trivial and faces three key challenges. First, existing KV compression methods are implemented as tightly coupled designs with incompatible code and parameter interfaces, making them difficult to reuse and compose into a plug-and-play interface. Second, abstracting KV compression into a searchable strategy space leads to an exponentially growing strategy space, making exhaustive profiling impractical. Third, online serving must meet quality and SLO budgets [35]; selecting strategies based solely on compression ratio or quality can be infeasible or suboptimal, and there is a lack of a constrained theoretical model to guide online selection and switching.

To address these challenges, we present *KVServe*. To the best of our knowledge, *KVServe* is the first *service-aware* and *adaptive* KV compression framework for disaggregated LLM serving. *KVServe* unifies KV compression techniques into a composable and extensible strategy space, senses online service context, and selects an optimal profile under quality and SLO constraints. Our key designs and contributions are:

- We abstract KV compression as a unified modular pipeline and decompose representative methods into pluggable components. Building on this abstraction, we introduce a new quantization component designed by us; through

cross-method composition and reuse, we form an enumerable and extensible strategy space.

- We design an efficient *Bayesian Profiling Engine*. Facing the combinatorial explosion of the strategy space, it uses Bayesian optimization to substantially reduce expensive end-to-end profiling runs, cutting offline search overhead from *1000 hours* to the *20-hour* scale.
- We propose a *Service-Aware Online Controller* that senses service context at runtime and rapidly selects the optimal profile from the offline candidates. The controller combines an analytical latency model with a lightweight bandit to correct mismatches between offline profiling and online execution, improving robustness to real-world drift.
- We integrate *KVServe* into the vLLM inference pipeline and evaluate it across many datasets, models, and GPU/network configurations. Compared with the baseline and SOTA KV compression methods, *KVServe* achieves up to $9.13\times$ JCT reduction in PD-separated serving, and up to $32.8\times$ TTFT reduction in KV-disaggregated serving.

2 BACKGROUND AND MOTIVATION

2.1 Bottleneck in Disaggregated LLM Serving

In recent years, the inference pressure of large language models has been driven by the dual scaling of *model size* and *context window*. Meanwhile, RAG and agentic workflows further push the demand for long-context online serving to accommodate more retrieved evidence and tool-call traces [2, 23]. Under this trend, production serving systems increasingly adopt *disaggregated* architectures (Fig. 2), by separating compute and KV state across different nodes and remote storage pools [33, 35, 49]. As a result, KV cache—previously resident in GPU memory—becomes an I/O payload that must be moved across devices over the network and moves onto the critical path of end-to-end latency.

Compute disaggregation: Prefill/Decode separation.

Prior separates prefill and decode across GPU nodes to reduce co-location contention and scale each stage independently. Prefill produces the prompt KV cache and ships it to decode, which consumes the KV during generation, enabling stage-aware placement on heterogeneous GPU pools. In practice, this split often breaks the shared high-speed interconnect domain (e.g., InfiniBand). With Ethernet-connected GPU nodes in the cloud, bandwidth limits can greatly amplify KV migration cost and make communication a dominant bottleneck. We quantify this on Llama-3.1 with Qasper, using H100 decode and varying prefill instances: Fig. 1 breaks down JCT into prefill, decode, and communication. At 10–50 Gbps, communication accounts for 16%–60% of JCT.

Cache Type	Accuracy				Compression Ratio				Metric Performance Rank
	GSM8K	HumanEval	Multi-News	Qasper	GSM8K	HumanEval	Multi-News	Qasper	
MixHQ	84.91	66.46	26.87	45.66	1.00	1.00	1.00	1.00	Best Acc
DuoAttn	83.47	62.80	26.08	44.46	5.85	3.98	6.20	6.65	
KIVI	81.35	60.37	26.44	44.97	4.32	2.61	4.45	4.89	
CacheGen	83.93	65.24	23.41	43.90	2.16	1.06	2.89	3.77	
Default	76.99	59.76	26.47	44.83	6.19	5.36	6.05	6.60	Best CR

Figure 3: Accuracy and compression ratio across workloads .

State disaggregation: KV cache offloading and cross-query reuse. In RAG, multi-turn conversations, and templated requests, systems often exploit *cross-query KV reuse* (e.g., prefix caching) to avoid redundant prefill, reducing TTFT and improving throughput. Keeping reusable KV resident in GPU memory is usually impractical: reuse can occur across requests far apart in time or on different GPU nodes, and GPU memory cannot hold many long-context KVs concurrently (often tens to hundreds of GB) [36]. As a result, systems offload KV to CPU/SSD tiers or a remote KV pool, but remote reads become latency-critical. Under 5–15Gbps links in typical cloud servers, KV communication accounts for up to 66% of end-to-end time [26], making KV movement a key bottleneck for latency and SLO attainment.

2.2 Rethinking KV Cache Compression: From Static to Service-Aware

In production LLM serving, requests are heterogeneous and are routinely *typed* by workload (e.g., math reasoning, code generation, long-document QA) via task- and intent-aware routing at the ingress, so that different request types can be steered to appropriate backends or execution paths (e.g., industry routers such as Red Hat’s *LLM Semantic Router* and NVIDIA’s *LLM Router*) [30, 32, 41]. Accordingly, we treat the workload label w for each session segment as a standard routing output of the serving stack (rather than a strong assumption), and focus on the service side: selecting a KV compression strategy conditioned on w and online conditions. Crucially, different workload types often tolerate different levels of quality loss (i.e., different quality budgets), and the serving environment further evolves over time.

Motivation 1: The Optimal KV Compression Strategy Varies Across Service Workloads. Existing KV compression methods are mostly *statically configured*: e.g., using a fixed transform, a fixed quantization granularity, and a fixed codec. Such methods may achieve favorable compression ratio and accuracy on certain workloads, but their advantages do not generalize well across workloads. The reason is that

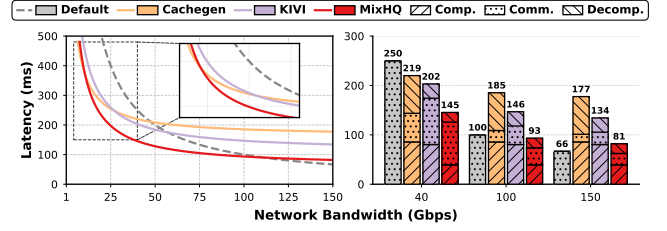


Figure 4: KV latency across effective bandwidths (left) and time breakdown (right).

different tasks exhibit substantially different request distributions and generation behaviors, which leads to systematic shifts in the statistics and compressibility of KV cache. As a result, the same compression strategy can yield markedly different accuracy and compression gains across tasks.

The results in Fig. 3 further validate this *workload dependence*. For example, KIVI achieves the best accuracy on Qasper, but ranks near the bottom on GSM8K and HumanEval. In contrast, DuoAttention performs best on GSM8K and HumanEval, yet performs worst on Multi-News and Qasper. Similar instability appears not only in accuracy but also in compression ratio. CacheGen reaches the best compression ratio of 6.20 $\times$ on Multi-News, but only 3.98 $\times$ on HumanEval, which is lower than MixHQ’s 5.36 $\times$. These observations can be summarized as follows: a static KV compression strategy cannot be optimal across diverse workloads.

TAKEAWAY-1

There is no universally optimal KV compression strategy across workloads. Practical systems must reason over *multiple candidate strategies* rather than committing to a single static configuration.

Motivation 2: The Optimal Strategy Also Depends on Bandwidth—and Can Even Hurt Performance. Beyond compression ratio, end-to-end speedup also depends on the service-side effective bandwidth and the compression/decompression throughput. For any compression strategy p , the KV latency has two parts: (i) communication of the compressed KV and (ii) compression and decompression. Comparing to uncompressed latency reveals speedup (or slowdown). Fig. 4 reports the KV latency of CacheGen, MixHQ, and KIVI across bandwidths. The optimal strategy switches with bandwidth: CacheGen is optimal at very low bandwidth, but as bandwidth increases it is overtaken by MixHQ and then KIVI (two intersections), with MixHQ best over a broad range.

More importantly, each profile is beneficial only within a bandwidth regime: once bandwidth exceeds a threshold, communication savings no longer offset (de)compression, making latency worse than no compression. In Fig. 4, the thresholds for the three methods are 50/55/110 Gbps, respectively. Therefore, if a system ignores bandwidth as a service

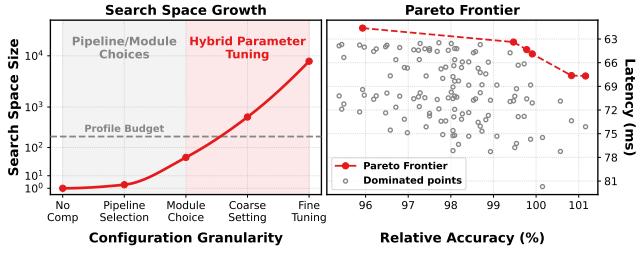


Figure 5: Left: Search space size under different granularities. Right: Latency-accuracy tradeoff of a collection of profiles from a representative pipeline.

state and applies a fixed static compression strategy, it cannot remain optimal across network conditions and may even directly hurt performance in some cases.

TAKEAWAY-2

The optimal KV compression strategy depends on dynamic service conditions (e.g., available bandwidth). A static, fixed strategy is unsafe in practice; KV compression must be *adaptive* and *service-aware* at runtime.

2.3 Challenges for Service-Aware KV Cache Compression

Challenge 1: The Combinatorial Explosion of the Strategy Space. To address the limitations of static configurations revealed by Motivation 1, one can abstract KV compression as a searchable strategy space of components and parameters, and then select the best configuration offline for a target workload. The core challenge is combinatorial explosion: as we move from *pipeline/module choices* to *fine-grained parameter tuning*, the number of candidates grows roughly exponentially with the degrees of freedom. Fig. 5 (left) shows that enabling fine-grained tuning quickly expands the space to nearly 10^4 candidates. Each candidate further requires an end-to-end profiling run (compression ratio, latency, and quality); in our setup this takes about 15 minutes, making exhaustive search cost tens to hundreds of GPU-hours—well beyond a practical offline budget. Therefore, our first challenge is to efficiently search this huge space while preserving candidate quality.

Challenge 2: The Latency-Quality Tradeoff without a Clear Decision Principle. Even after offline profiling compresses the space into a finite candidate set, online selection still faces an inherent latency-quality trade-off with no single metric that resolves it. Fig. 5 (right) plots 131 candidates under the same workload and shows a highly dispersed distribution: latency can differ markedly at similar quality levels, and further latency reductions often incur non-trivial quality loss. Hence, a production system must choose a feasible and optimal strategy under constraints such as SLO and an accuracy budget; ranking by compression ratio alone or

quality alone can frequently yield infeasible or suboptimal profiles. This motivates a constrained model that jointly captures (de)compression overhead, post-compression volume, and quality degradation, enabling interpretable selection and switching as service conditions change.

3 PROBLEM FORMULATION

3.1 Serving System Model

We consider two common KV-movement paths in *disaggregated LLM serving*: (i) prefill→decode migration under *PD separation*, and (ii) fetching/offloading KV under *KV state offloading/reuse*. In both cases, KV becomes an explicit payload that crosses a network/I/O boundary and contributes directly to end-to-end latency. We therefore use a *request* as the decision granularity: the system selects a compression profile when the request’s KV movement begins and keeps it consistent throughout the request. Crucially, the realized communication cost is governed by the *effective* network/I/O regime—application-level goodput under contention—rather than nominal link bandwidth. Accordingly, we incorporate lightweight runtime communication signals into the service context to enable network-aware, constraint-driven profile selection within each request. The service context within this window is abstracted as:

$$c = (w, B, T_{\text{SLO}}, q_{\min}),$$

where w denotes the workload class of the session segment (provided by an upper-layer router/classifier; we do not study its implementation), B is the currently available *effective bandwidth* (a unified abstraction of network or I/O goodput), T_{SLO} is the latency budget for the session segment, and $q_{\min}$ is the minimum quality requirement.

A KV compression strategy (profile) can be represented by a parameterized triple:

$$p = (cr_p, s_p, q_p),$$

where cr_p is the compression ratio, defined as $cr_p \triangleq \frac{V}{V_p}$, with V being the total amount of uncompressed KV to be moved within the session segment (in bytes) and V_p being the total compressed KV size under strategy p . s_p is the effective (de)compression throughput (bytes/s), defined as the harmonic mean of the encoding throughput s_p^{enc} and the decoding throughput s_p^{dec} :

$$s_p \triangleq \left(\frac{1}{s_p^{\text{enc}}} + \frac{1}{s_p^{\text{dec}}} \right)^{-1} = \frac{s_p^{\text{enc}} s_p^{\text{dec}}}{s_p^{\text{enc}} + s_p^{\text{dec}}},$$

so that the total encoding and decoding time can be written as $\frac{V}{s_p^{\text{enc}}} + \frac{V}{s_p^{\text{dec}}} = \frac{V}{s_p}$. Finally, q_p denotes the quality metric of strategy p under workload w (e.g., task accuracy or an equivalent measure of quality loss).

Given a dynamic service context c , our goal is to select a strategy p for each session segment that satisfies the service

requirements T_{SLO} and $q_{\min}$ while optimizing end-to-end performance; the latency model and the resulting optimization problem are presented in the next section.

3.2 Constrained Optimization

Within each *session segment* decision window, we use the segment-level end-to-end completion time, Job Completion Time (JCT), as the optimization target. We decompose it into two parts: (i) the model execution cost that is independent of the KV compression strategy, and (ii) the additional cost introduced by KV (de)compression and KV movement. Let V denote the total amount of *uncompressed* KV that must cross the boundary within the segment (in bytes), and let B denote the *effective bandwidth* (bytes/s) observed by KV movement during online serving. Let $T_{\text{model}}(w)$ denote the model execution cost under workload class w , which is approximately invariant to the choice of compression strategy given a fixed model and serving configuration; we also absorb other strategy-independent operator execution and scheduling overheads into $T_{\text{model}}(w)$.

For any compression strategy $p = (cr_p, s_p, q_p)$, the compressed KV volume is $V_p = \frac{V}{cr_p}$. Using the definition of the effective (de)compression throughput s_p from 3.1, we model the segment JCT as:

$$T_p(c) = T_{\text{model}}(w) + \frac{V}{s_p} + \frac{V}{B cr_p}, \quad T_0(c) = T_{\text{model}}(w) + \frac{V}{B}. \quad (1)$$

Here, $\frac{V}{s_p}$ represents the sum of encoding and decoding time. We assume that the amount of data processed by (de)compression is of the same order as the KV volume to be moved, and we include operator execution and scheduling overheads unrelated to KV (de)compression in $T_{\text{model}}(w)$.

Online strategy selection under service context c must satisfy the segment-level latency budget and the minimum quality requirement, and we select a profile to minimize $T_p(c)$ under these requirements. For convenience, we define the feasible set of strategies under context c as

$$\mathcal{P}(c) \triangleq \{p \in \mathcal{P} \mid T_p(c) \leq T_{\text{SLO}}, q_p(w) \geq q_{\min}\}, \quad (2)$$

where $\mathcal{P}$ is the set of selectable compression strategies. We then formulate the segment-level strategy selection as the following constrained optimization problem:

$$p^*(c) \in \arg \min_{p \in \mathcal{P}(c)} T_p(c). \quad (3)$$

This formulation explicitly captures the joint effect of four factors: the effective bandwidth B determines the upper bound of time savings from compression, the effective throughput s_p determines the additional (de)compression overhead, the compression ratio cr_p determines the red KV volume after compression, and $q_p(w)$ captures the quality cost. In the following sections, we derive benefit conditions

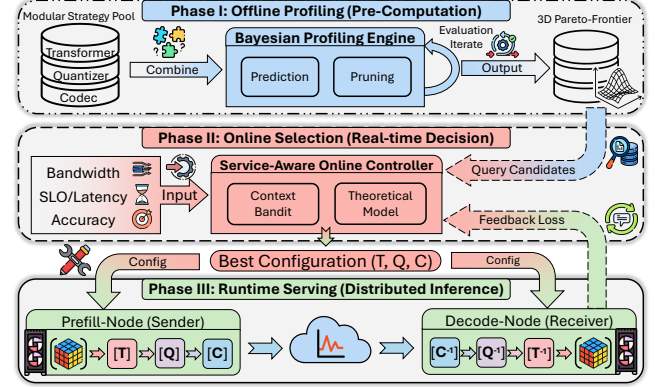


Figure 6: Overview Architecture of KVServe.

from this model and design a policy that selects and switches strategies in response to changing conditions.

4 DESIGN OVERVIEW

To address the above problems and challenges, we propose KVServe. To the best of our knowledge, KVServe is the first *service-aware* and *adaptive* KV communication compression framework for *disaggregated LLM serving*. Unlike prior approaches that rely on static configurations to optimize a single metric, KVServe unifies mainstream KV compression techniques into a composable and extensible strategy space, and adapts to online service conditions to select the optimal KV compression strategy. Under SLO and quality constraints, KVServe aims to minimize end-to-end latency. KVServe consists of three core components (shown in Fig. 6):

- **Modular Strategy Pool.** We abstract KV compression as a modular pipeline composed of pluggable components, and map representative existing methods into this abstraction. Beyond incorporating improved variants of existing components, we also enable new components to be designed and integrated, forming an enumerable space.
- **Bayesian Profiling Engine.** Facing the combinatorial explosion of the strategy space, the profiling engine uses Bayesian Optimization with Gaussian Processes to substantially reduce the number of expensive end-to-end profiling runs. It ultimately derives a candidate set defined by a 3D Pareto frontier for fast online selection.
- **Service-Aware Online Controller.** During online inference, the controller senses the service context and selects the optimal profile from the offline candidate set. It has two layers: (i) an analytical latency model that provides interpretable end-to-end benefit estimates and derives benefit boundaries; and (ii) a lightweight online bandit that refines decisions based on runtime observations, correcting system drift and improving robustness.

Overall, KVServe operates in three stages: *Offline Profiling*, *Online Selection*, and *Runtime Serving*. In *Offline Profiling*, the

Bayesian Profiling Engine efficiently searches the *Modular Strategy Pool* and constructs the candidate set. In *Online Selection*, the *Service-Aware Online Controller* chooses the most suitable compression profile given the current service state and constraints. Finally, in *Runtime Serving*, KVServe executes the selected strategy at KV movement boundaries.

5 OFFLINE PROFILING ENGINE

5.1 Constructing the strategy space

Existing KV cache optimizations—including rotation [3], quantization [28], and entropy coding [47]—are predominantly studied in isolation, often yielding suboptimal trade-offs between compression ratio (CR) and accuracy (Acc). To bridge this gap, we propose a generalized *KV Cache Compression Pipeline* that unifies these disjoint strategies into a composable framework, recasting compression as a search problem over a comprehensive strategy space.

Pipeline Abstraction and Module Instantiation. We formalize the KV cache compression lifecycle as a sequential composition of three distinct stages, $BS = C(Q(\mathcal{T}(X)))$, as schematically illustrated in Fig. 7:

❶ **Transformer ($\mathcal{T}$):** A pre-processing stage reshaping distributions to facilitate downstream compression. Modules include Delta [26], Hadamard [3] and Affine [29].

❷ **Quantizer (Q):** The primary stage for bit-width reduction. This module encompasses multi-dimensional quantization methods [28] and supports *Mixed-Precision Quantization* at both layer-wise and head-wise granularities.

❸ **Codec (C):** The final stage encodes the data stream to minimize footprint. We integrate the high-performance library nvCOMP [31] library to support efficient algorithms.

By decomposing existing SOTA methods into these atomic components, KVServe enables the exploration of their Cartesian product. This extensible architecture allows for arbitrary combinations (e.g., pairing a QuaRot transformer with a CacheGen quantizer) to identify synergistic configurations that outperform isolated baselines.

Mixed-Precision Head-Wise Quantization (MixHQ). In this pipeline, we also propose a novel framework for Q that shifts the paradigm from binary pruning [43] to variable

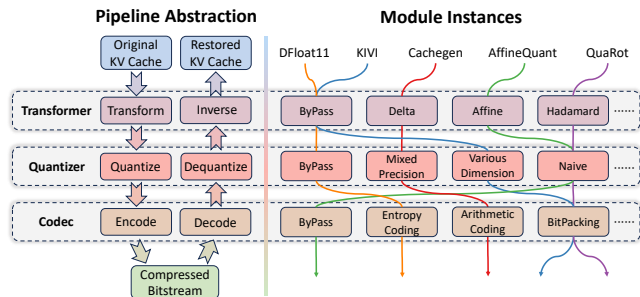


Figure 7: The Unified KV Cache Compression Pipeline.

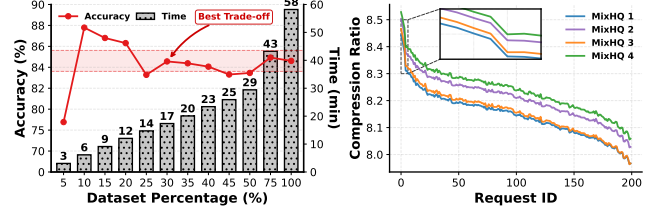


Figure 8: Profiling Efficiency and Ranking Consistency.

precision allocation. By distinguishing between Retrieval Heads and Streaming Heads, MixHQ applies aggressive ultra-low bit-width quantization to the latter instead of discarding them, while retaining Retrieval Heads in high precision to preserve critical long-range dependencies.

Crucially, this framework is orthogonal to the granularity of importance estimation. It supports seamless generalization to the layer dimension (assigning lower bit-widths to deeper layers like PyramidKV [46]) and the token dimension (preserving heavy-hitters like SnapKV [22]). This flexibility enables integration with various importance scoring methods, effectively transforming discrete pruning decisions into a continuous spectrum of precision allocation.

5.2 Bayesian Profiling Engine

5.2.1 Profiling Analysis and Optimization Strategy. To identify the optimal pipeline balancing CR and Acc, we must navigate a massive combinatorial strategy space $\mathcal{S}$. As illustrated in *Motivation 1* (Fig. 5 left), the search space grows exponentially as configuration granularity deepens from *Pipeline/Module Choices* to *Hybrid Parameter Tuning*. This explosive complexity renders brute-force methods impractical, necessitating a highly automated search strategy.

Through empirical analysis, we derive two critical observations guiding our engine design:

Observation 1: High Cost of Acc Evaluation. Fig. 8 (left) reveals that executing full-dataset inference is prohibitively expensive. However, accuracy on uniformly sampled subsets stabilizes quickly, approximating full performance with negligible error. Thus, we employ sampled data as a reliable proxy to accelerate profiling.

Observation 2: Stability of CR Relative Rankings. Although absolute compression ratios fluctuate with content, Fig. 8 (right) demonstrates that the relative ranking of configurations remains strictly invariant across requests, even among MixHQ candidates with highly proximate ratios. This stability ensures that high-performing configurations identified offline reliably translate to online optimality.

Guided by these insights, we formulate the task as a *Constrained Black-Box Optimization* problem. We adopt *Bayesian Optimization (BO) with Gaussian Processes (GP)* over evolutionary algorithms or random search for two reasons: (i)

Algorithm 1: Constraint-Aware Bayesian Optimization with Gaussian Processes

Input: Strategy Space $\mathcal{S}$; Accuracy Thres Acc_{ths} ; Pruning Buffer ϵ ; Max Iterations T_{max} ;
Output: Feasible configuration set $\mathcal{F}$;

```

1  $S_{emb} \leftarrow \text{OneHot}(S_{cat}) \cup \text{MinMax}(S_{num})$ 
2 Initialize GP Model  $\mathcal{M}_{GP}$  and Observation Set  $\mathcal{D}$ 
3 for  $t \leftarrow 1$  to  $T_{max}$  do
4   Fit  $\mathcal{M}_{GP}$  on  $\mathcal{D}$ 
5    $\lambda \leftarrow \text{GetExplorationWeight}(t)$ 
6    $c_{curr} \leftarrow \text{argmax}_{c \in S_{emb}} \text{AF}(\mathcal{M}_{GP}, c, \lambda)$ 
7    $Acc_{curr}, CR_{curr} \leftarrow \text{Evaluate}(c_{curr})$ 
8    $\mathcal{D} \leftarrow \mathcal{D} \cup \{(c_{curr}, CR_{curr}, Acc_{curr})\}$ 
9   if  $Acc_{curr} \geq Acc_{ths}$  then
10     $S_{emb} \leftarrow S_{emb} \setminus \{c \mid CR(c) < CR_{curr} - \epsilon\}$ 
11     $\mathcal{F} \leftarrow \mathcal{F} \cup \{c_{curr}\}$ 
12   else if  $Acc_{curr} \ll Acc_{ths}$  then
13     $S_{emb} \leftarrow S_{emb} \setminus \{c \mid CR(c) > CR_{curr} + \epsilon\}$ 
14    $k_{fail} \leftarrow \text{UpdateFailureTimes}(Acc_{curr}, Acc_{ths}, t)$ 
15   if  $\text{CheckEarlyStopping}(S_{emb}, \mathcal{D}, k_{fail})$  then
16     break
17 return  $\mathcal{F}$ 

```

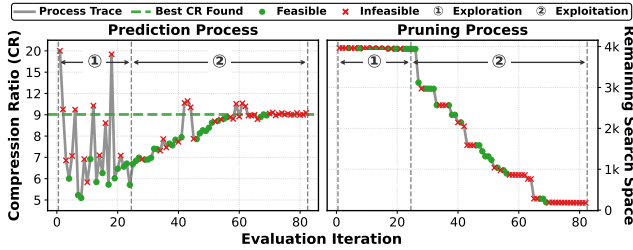


Figure 9: Prediction and Pruning Process Visualization.

sample efficiency—given the high evaluation cost, minimizing the number of iterations is crucial, and BO leverages a surrogate model to approach the global optimum with far fewer samples; and (ii) *uncertainty modeling*—GPs estimate both the mean and variance, enabling principled *exploration-exploitation* trade-offs and reducing the risk of getting trapped in local optima.

Formally, given a configuration $c \in \mathcal{S}$, we solve:

$$\max_c CR(c) \quad \text{s.t.} \quad Acc(c) \geq Acc_{threshold},$$

where $CR(c)$ and $Acc(c)$ denote the compression ratio and model accuracy, respectively.

5.2.2 Bayesian Optimization with Gaussian Process. Our profiling engine operates on a Bayesian Optimization cycle. It iteratively models the configuration-to-accuracy mapping using a Gaussian Process Surrogate Model and selects the next candidate to evaluate based on a utility score, repeating this process until convergence or early stopping.

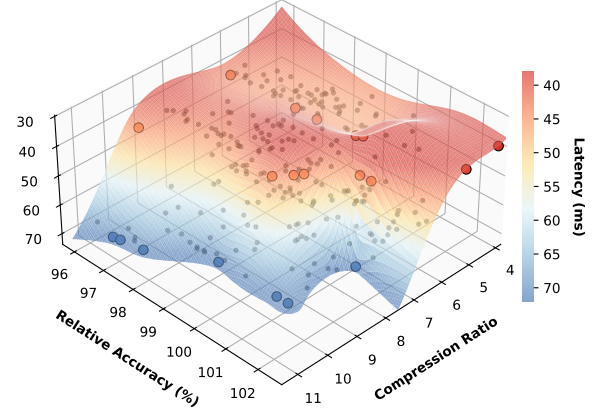


Figure 10: The 3D Pareto Frontier of the Strategy Spaces.

Acquisition Function (AF). To guide the selection, we design a custom utility $\alpha(c)$ that balances maximizing the expected CR within constraints (*Exploitation*) against reducing uncertainty (*Exploration*):

$$\alpha(c) = \underbrace{CR(c) \cdot P(\text{Feasible})}_{\text{Exploitation}} + \underbrace{\lambda_t \cdot \sigma_{norm}(c)}_{\text{Exploration}}, \quad (4)$$

where $P(\text{Feasible})$ is the probability of satisfying the accuracy constraint derived from the GP posterior, and λ_t is an exploration weight that decays over iterations t .

However, generic BO is ill-suited for our heterogeneous search space (mixed categorical/continuous parameters) and strict offline time constraints. Unlike standard asymptotic convergence, we require pinpointing optimal configurations within extremely limited iterations. Consequently, we enhance the architecture with specific optimizations for *prediction* and *pruning*, as detailed in Alg. 1.

Heterogeneous-Parameter Encoding (Line 1). To resolve metric incompatibility in our mixed-parameter space, we map categorical and numerical variables to a unified embedding S_{emb} via One-Hot and Min-Max scaling, ensuring the GP kernel correctly measures structural similarity.

Exploration-Exploitation Strategy (Lines 5-6). We employ a dynamic strategy where the exploration weight λ_t decays exponentially. This transitions the search from global exploration (high uncertainty sampling) to rapid exploitation (convergence on optima), as visualized in Fig. 9 (left).

Bi-Directional Pruning (Lines 9-13). Leveraging the monotonic CR-Acc trade-off, we prune bi-directionally during exploitation, as shown in Fig. 9 (right): discarding higher-CR candidates if infeasible, and lower-CR ones if feasible, focusing solely on maximizing compression.

Early-Stopping Mechanism (Lines 14-16). To minimize overhead, the engine terminates execution early if consecutive failures k_{fail} exceed a pre-defined limit or if the effective search space is exhausted.

The efficacy of these strategies is visualized in Fig. 9. While an exhaustive search of over 4,000 candidates would require $\sim 1,000$ hours, our algorithm converges in fewer than 80 iterations (~ 20 hours). This represents a $50\times$ reduction in profiling overhead, effectively transforming an intractable exponential search into a manageable offline task.

5.2.3 Outcome: The 3D Pareto Frontier. The output $\mathcal{F}$ from Alg. 1 contains all feasible history, yet many configurations are dominated. Furthermore, maximizing CR alone is insufficient in networked serving, as computational overhead can negate communication gains. To address this, we introduce *Latency* as a third critical dimension for evaluation.

We compute the *3D Pareto Frontier* by projecting $\mathcal{F}$ into Acc-CR-Lat space, retaining only non-dominated points. As shown in Fig. 10, the resulting surface represents optimal trade-offs among quality, footprint, and delay.

This *3D Pareto Frontier* serves as a static runtime lookup table. It provides a candidate set for the *Online Selection* (Sec. 6) to choose optimal strategies under dynamic context like bandwidth and SLO constraint.

6 SERVICE-AWARE ONLINE CONTROLLER

Using the profiling engine in Sec. 5, we shrink the massive strategy space into a finite 3D Pareto candidate set. However, a candidate set alone is insufficient in production. The system must sense online context (e.g., bandwidth, SLO, and quality budget) and select the latency-minimizing compression strategy with negligible overhead, while remaining robust to offline-to-online drift. To this end, we introduce a *Service-Aware Online Controller*. The controller is built on an interpretable analytical latency model and further corrects runtime perturbations via a lightweight learnable bandit.

6.1 Analytical Model

In disaggregated LLM serving, KV movement occurs at clear system boundaries, such as prefill $\rightarrow$ decode migration in PD separation or fetching from a remote KV pool. We use a *request* as the decision granularity: the system selects a profile p at the start of KV movement and keeps it fixed for the request. Given context $c = (w, B, T_{\text{SLO}}, q_{\text{min}})$, and under a workload w the request JCT follows Eq. (1), subject to latency and quality budgets.

To ensure that the chosen profile meets the quality requirement, we bucket profiles by accuracy loss and restrict selection to the bucket matching the request's quality budget. After fixing a quality bucket b , the key variables for online selection reduce to each profile's compression ratio cr_p and effective throughput s_p . We first ask a fundamental question: *when does compression actually yield end-to-end speedup?* Using the latency model in Sec. 3.1, by comparing $T_p(c)$ with

$T_0(c)$, we can express a benefit condition: $T_0(c)/T_p(c) > 1$. This leads to a bandwidth-threshold condition in Eq. (5).

$$B_p^* \triangleq \left(1 - \frac{1}{cr_p}\right) s_p, \quad T_p(c) < T_0(c) \iff B < B_p^*. \quad (5)$$

Notably, we observe that the condition is independent of the KV volume V and depends only on the compression ratio and (de)compression throughput; moreover, the condition collapses to a threshold on the effective bandwidth B . This yields our first theorem.

THEOREM 6.1 (BENEFIT CONDITION: BANDWIDTH THRESHOLD). *For any profile p , its offline parameters (e.g., cr_p and s_p) determine a bandwidth threshold B_p^* (Eq. (5)). The profile is beneficial if $B < B_p^*$; otherwise it is non-beneficial and can be filtered online, substantially shrinking the candidate set.*

After filtering, we further use the latency model to characterize which profile is optimal under a given bandwidth. For a workload w and quality bucket b , we minimize T_p over the feasible set $\mathcal{P}_b(w)$. For analysis, we let $x = 1/B$ and rewrite T_p as a linear function of x ,

$$\tilde{T}_p(x) = \frac{T_p(c) - T_{\text{model}}(w)}{V} = \frac{1}{s_p} + \frac{1}{cr_p} x, \quad x = \frac{1}{B}. \quad (6)$$

This yields the following structural result.

THEOREM 6.2 (PIECEWISE-OPTIMAL POLICY). *For a workload w and quality bucket b , minimizing $\tilde{T}_p(x)$ over $p \in \mathcal{P}_b(w)$ is equivalent to taking the lower envelope of the lines $\{\tilde{T}_p(x)\}$. Hence the optimal profile is piecewise constant in $x = 1/B$: there exist breakpoints $0 = x_0 < x_1 < \dots < x_m$ such that for any $x \in [x_i, x_{i+1})$, the optimal profile is $p_i \in \mathcal{P}_b(w)$.*

Together, Theorems 6.1 and 6.2 provide an efficient and interpretable baseline selection mechanism. Offline, we construct the lower envelope in each quality bucket and obtain a piecewise policy table. Online, given the measured bandwidth B , we first apply Theorem 6.1 to filter obviously non-beneficial profiles, yielding a tighter candidate set. Then, by Theorem 6.2, we only need to look up the interval for $x = 1/B$ to return the optimal profile p_i , and simultaneously return the neighboring profiles as a candidate set. This analytical mechanism achieves $O(1)$ decision cost, but it may still be affected by online drift. Next, we introduce lightweight online learning to perform residual correction for the mismatch between offline profiling and real serving conditions.

6.2 Residual-Corrected Bandit

In online serving, parameters estimated from offline profiling often drift from reality. For example, GPU load and queue contention change the actual (de)compression throughput, and system scheduling and concurrency introduce additional overhead. As a result, the analytical model's latency predictions can deviate from runtime observations. Relying solely

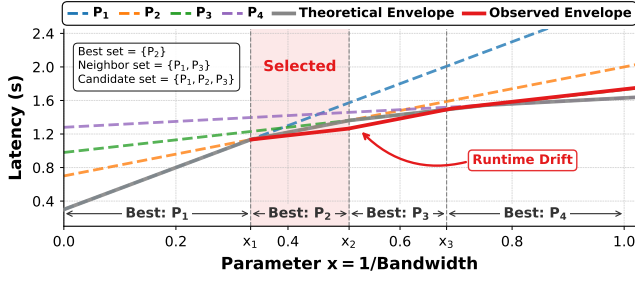


Figure 11: Candidate set generation and bandit-based residual correction on the lower envelope.

on offline parameters may cause the system to deviate from the true optimum during certain periods and require frequent re-profiling or manual retuning.

To address this, we add an extremely lightweight online learning layer on top of the analytical model to perform residual correction (Fig. 11). The key idea is that the analytical model provides a strong prior by proposing the best profiles for the current bandwidth interval, and the online bandit only learns the difference between model prediction and runtime observation, achieving robustness at low cost.

Theorem 6.2 shows that the optimal policy is piecewise constant in $x = 1/B$; under mild online drift, the most likely change is that the optimal choice switches among adjacent segments of the lower envelope. Therefore, we construct a tiny candidate set centered on the model-optimal profile $p_{b,i}^{\text{model}}$ for bucket b and interval i , augmented by 1–2 neighboring profiles on the envelope: $p_{b,i}^{\text{cand}} = \{p_{b,i}^{\text{model}}\} \cup \text{Nbr}(p_{b,i}^{\text{model}})$. The candidate set is small (typically 2–3 profiles), keeping exploration cost bounded. We treat each pair (b, i) as an independent small environment and perform online learning only within $p_{b,i}^{\text{cand}}$.

The goal of online learning is not to re-fit the full latency model, but to learn residuals relative to the analytical prediction. For any candidate profile $p \in p_{b,i}^{\text{cand}}$, the analytical model predicts $\text{JCT}_{\hat{p}}(c)$. Let the observed request JCT be T^{obs} ; we define the residual as $\delta \triangleq T^{\text{obs}} - \hat{T}_{\hat{p}}(c)$. For each candidate, we maintain an exponentially weighted moving average (EWMA) residual estimate $\bar{\delta}_{b,i}(p)$ and a usage count $N_{b,i}(p)$. After each execution, we update the residual by

$$\bar{\delta}_{b,i}(p) \leftarrow (1 - \alpha)\bar{\delta}_{b,i}(p) + \alpha\delta, \quad (7)$$

where $\alpha \in (0, 1]$ controls tracking speed under non-stationary drift. Given $\bar{\delta}_{b,i}(p)$, the corrected effective latency is

$$T_p^{\text{eff}} = \hat{T}_p(c) + \bar{\delta}_{b,i}(p). \quad (8)$$

We perform ϵ -greedy selection over $p_{b,i}^{\text{cand}}$: with probability $1 - \epsilon$, we choose the profile that satisfies constraints and minimizes T_p^{eff} ; with probability ϵ , we randomly explore among

the remaining candidates. Because the action space per environment is at most three profiles, we do not need heavier contextual bandits (e.g., LinUCB) to achieve fast adaptation.

Online exploration carries the primary risk of SLO violations, so we enforce safety guardrails. First, we use $\hat{T}_p(c) \leq T_{\text{SLO}}$ as a conservative feasibility filter; if the feasible set is empty, we fall back to a default conservative compression configuration. Second, we use a cooldown mechanism for unpredicted violations: for each profile we track recent SLO violations, and if a profile exceeds K violations in the most recent M uses, we temporarily remove it from the candidate set during a cooldown window to reduce repeated risk.

This online learning layer incurs negligible overhead: each request it evaluates at most 2–3 candidates and updates constant-size state, making it safe to deploy in the control plane without affecting token-level inference latency. Combined with the analytical model, the residual-corrected bandit enables KVServe to perform stable service-aware strategy selection under constraints and to sustain near-optimal end-to-end speedup under serving perturbations.

7 EVALUATION

In this section, we structure our analysis to address the following key research questions:

- **End-to-End Performance:** How much does KVServe reduce the end-to-end completion time, compared to baselines under varying conditions? (Sec. 7.2)
- **Pareto Efficiency:** Can our offline search algorithm effectively identify the optimal compression pipelines that balance high CR with strict Acc constraints? (Sec. 7.3)
- **Algorithmic Effectiveness:** How do the specific optimizations in our offline search and online decision modules contribute to the overall system performance? (Sec. 7.4)

7.1 Experimental Setup

We implement KVServe atop vLLM 0.10.1 [20], extending its architecture to support disaggregated prefill-decode execution with our compression pipeline injected into the communication path. Additionally, we integrate the lm-eval-harness [13] directly into the system to evaluate the accuracy impact of KV compression during online inference across *PD Separation* and *Prefix Caching* scenarios.

Models and Datasets. We evaluate our system using Qwen2.5-7B-Instruct [40, 45], Llama-3.1-8B-Instruct [14], and the larger Qwen2.5-72B-Instruct. Our dataset is designed to verify both search effectiveness and generalization capability: (i) *Profiling Datasets:* We search the Pareto Frontier using four datasets: GSM8K [8] (Math), HumanEval [5] (Code), Multi-News [4] (Summarization), and Qasper [4] (QA). (ii) *Unseen Datasets:* To evaluate ability, we use 2WikiMQA and

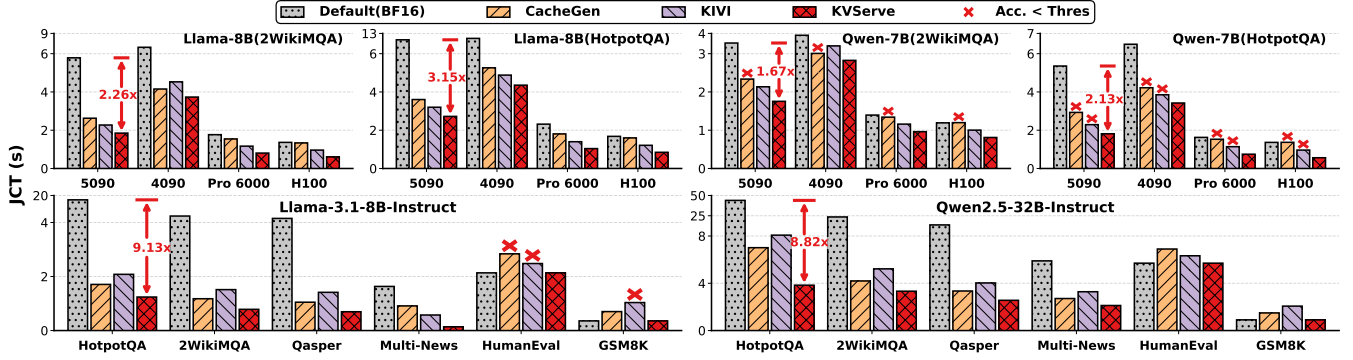


Figure 12: End-to-End Performance across Hardware and Workloads. Top row evaluates JCT scalability across hardware tiers; bottom row benchmarks diverse datasets. Crosses (x) indicate configurations failing the 97% relative accuracy threshold.

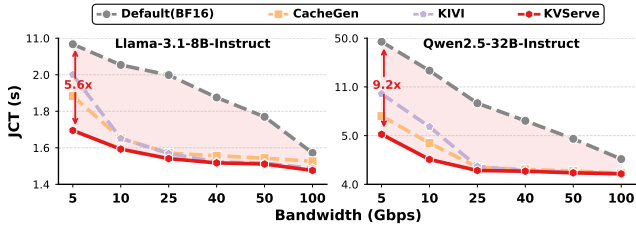


Figure 13: JCT in PD Separation.

HotpotQA [4]. These remain unseen during profiling to verify generalization to new tasks.

Baselines. We compare KV Serve against three optimizations, integrating core algorithms of CacheGen and KIVI as pipeline modules for comparison: (i) *CacheGen* [26]: adapts compression by tuning quantization granularity within a fixed pipeline. (ii) *KIVI* [28]: A static method applying fixed asymmetric 2-bit quantization regardless of context. (iii) *DuoAttention* [43]: A pruning-based method benchmarking token dropping against our mixed-precision approach.

Testbed. We conduct offline profiling on 4x A100 (40GB) GPUs and use H100 for decoding. The prefill nodes cover three tiers with distinct network bandwidths: (i) *Consumer Grade (10 Gbps)*: 2x RTX 4090 (24GB) and 2x RTX 5090 (32GB). (ii) *Workstation Grade (50 Gbps)*: 2x RTX Pro 6000 (96GB). (iii) *Data-Center Grade (100 Gbps)*: 2x H100 (80GB).

7.2 End-to-End Performance

We evaluate KV Serve’s JCT across diverse hardware and network configurations. Benchmarking against SOTA baselines highlights its efficiency in mitigating communication bottlenecks while preserving accuracy.

System Performance Across Diverse Hardware and Workloads. To assess the end-to-end performance of KV Serve in practical deployment, we evaluate the JCT across a wide range of hardware tiers and diverse task categories.

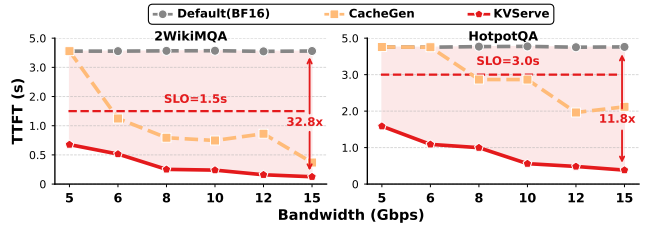


Figure 14: TTFT in Prefix Caching.

As shown in the top row of Fig. 12, we evaluate performance across diverse prefill hardware tiers. KV Serve consistently achieves the lowest JCT, delivering up to 3.15x speedup on bandwidth-constrained devices. Crucially, on Qwen2.5-7B-Instruct, static baselines like CacheGen and KIVI frequently violate the 97% relative accuracy threshold (marked by x), whereas KV Serve strictly maintains precision while outperforming them. Even in high-bandwidth environments, KV Serve avoids the significant decompression bottlenecks that plague static methods, ensuring robust performance where others often underperform.

The robustness of our system is further validated across diverse datasets using Llama and Qwen, as illustrated in the bottom row of Fig. 12. KV Serve consistently yields the lowest JCT, achieving drastic reductions on long-context tasks (e.g., 9.13x on HotpotQA). A critical advantage is observed on short-context workloads like GSM8K and HumanEval, where the computational overhead of (de)compression outweighs communication savings, causing baselines to suffer negative optimization (higher JCT than Default). KV Serve’s service-aware controller correctly anticipates this trade-off and bypasses compression by filtering non-beneficial profiles via theoretical modeling, ensuring performance converges to the uncompressed baseline rather than degrading it.

Adaptive Performance Across Network Bandwidths and Serving Scenes. To evaluate the system adaptability under fluctuating network conditions, we analyze KV Serve

Table 1: Accuracy and Compression Efficiency. Evaluated on Qwen2.5-7B-Instruct via offline Pareto search on A100 under a 97% relative accuracy constraint. Cell values denote Accuracy / Compression Ratio; bold indicates accuracy exceeding the baseline. Average Accuracy reports the relative percentage against Default.

Method (Acc / CR)	Profiling Workloads				Unseen Workloads		Average (Rel. Acc / CR)
	GSM8K	HumanEval	Multi-News	Qasper	2WikiMQA	HotpotQA	
Default (BF16)	82.64 / 1.00	83.54 / 1.00	23.73 / 1.00	43.34 / 1.00	46.96 / 1.00	57.53 / 1.00	100.00 / 1.00
CacheGen	72.55 / 6.01	57.32 / 4.06	17.95 / 6.33	25.95 / 6.81	28.53 / 6.84	24.09 / 6.94	65.76 / 6.17
KIVI	81.50 / 4.26	81.71 / 2.49	23.38 / 4.50	41.05 / 4.96	46.33 / 5.04	55.37 / 5.15	97.43 / 4.40
DuoAttention	82.56 / 2.21	82.93 / 1.06	20.43 / 2.92	40.53 / 3.83	45.45 / 4.08	56.00 / 4.50	95.48 / 3.10
KV Serve-Unified	81.50 / 7.07	84.15 / 6.20	23.18 / 7.36	42.35 / 7.85	47.04 / 7.94	54.23 / 8.07	98.20 / 7.42
KV Serve-Aware	84.53 / 7.29	84.15 / 6.04	24.75 / 10.12	43.48 / 8.60	46.32 / 8.72	55.11 / 8.90	100.35 / 8.28

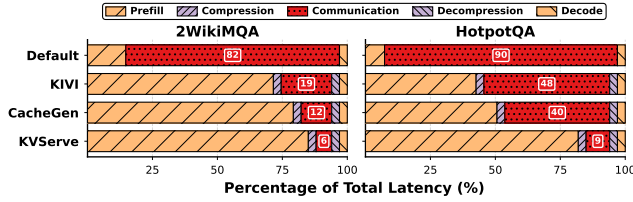


Figure 15: Latency Breakdown across Inference Stages.

across two representative disaggregated scenarios: *PD Separation* and state-offloading with *Prefix Caching*. We enforce target bandwidths via sender-side rate control using Linux traffic shaping and NIC-level rate limiting for RoCE.

As illustrated in Fig. 13, we first evaluate the end-to-end JCT in the *PD Separated* serving scenario for Llama-3.1-8B-Instruct and Qwen2.5-32B-Instruct on the 2WikiMQA dataset using an Pro 6000 prefill node. Testing across bandwidths from 5 to 100 Gbps, the red shaded area highlights KV Serve’s substantial acceleration over the Default(BF16). Under constrained bandwidth (5 Gbps), KV Serve delivers up to 9.2× speedup. Notably, as bandwidth increases, KV Serve maintains the optimal lower bound by dynamically selecting lower-overhead strategies, effectively avoiding the negative optimization observed in static baselines.

Beyond PD Separation setups, KV Serve also excels in state-disaggregated scenarios leveraging *Prefix Caching* on remote KV pools. Fig. 14 depicts the Time To First Token (TTFT) for Qwen2.5-32B-Instruct on the 2WikiMQA and HotpotQA datasets using Pro 6000 node. We benchmark against CacheGen, which dynamically falls back to costly re-computation if it cannot meet the target SLO. As observed at lower bandwidths (5–6 Gbps), CacheGen fails to find a valid configuration and degrades to the Default baseline’s high latency. In contrast, KV Serve consistently satisfies strict SLO constraints across the entire 5–15 Gbps range by instantly pinpointing optimal profiles from its Pareto frontier. This transforms otherwise infeasible fetches into valid cache hits, achieving a peak speedup of 32.8× over re-computation.

Latency Breakdown Analysis. To pinpoint the source of performance gains, we decompose the end-to-end latency

into five stages—Prefill, Compression, Communication, Decompression, and Decode—using Qwen2.5-32B-Instruct on 2WikiMQA and HotpotQA. As shown in Fig. 15, the Default baseline is severely network-bound, with communication consuming 82–90% of the total JCT. KV Serve effectively neutralizes this bottleneck, slashing the communication share to a mere 6–9%, significantly outperforming baselines like KIVI and CacheGen in HotpotQA. The online control overhead is negligible: each decision takes < 1 ms. Crucially, the added computational overhead for compression and decompression remains negligible, successfully shifting the system profile from network-bound back to compute-bound.

7.3 Accuracy and Compression Ratio

We evaluate the quality of the compression configurations identified by our Bayesian Profiling Engine using Qwen2.5-7B-Instruct in modular pipeline. Specifically, we select the configuration that maximizes the compression ratio subject to a strict accuracy preservation constraint. Tab. 1 reports the Acc and CR across four profiling workloads and two unseen workloads. We compare two variants of our approach: *KV Serve-Unified*, which searches for a default robust configuration using a mixed dataset of the four profiling workloads, and *KV Serve-Aware*, which performs independent searches for each workload to identify the optimal configuration. For unseen workloads, *KV Serve-Unified* applies the configuration derived from the mixed profiling workloads, whereas *KV Serve-Aware* adopts the *Qasper*-specific configuration due to their shared QA task alignment.

Existing methods struggle to maintain high Acc and CR on Qwen2.5-7B-Instruct. CacheGen exhibits substantial accuracy collapse across most datasets (e.g., 57.32% on HumanEval). We attribute this to its uniform quantization; unlike Llama3, the Qwen2.5 architecture includes bias terms in Key/Value projections, resulting in a non-zero-centered, non-symmetric distribution that is ill-suited for uniform mapping. KIVI, while maintaining better stability than CacheGen, hits a compression ceiling. Although its 2-bit quantization

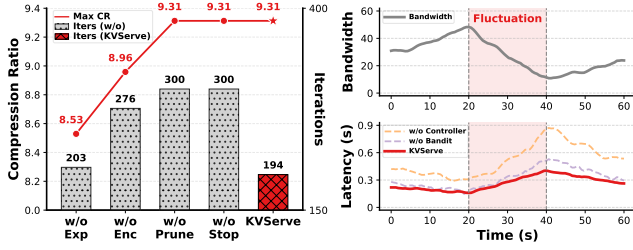


Figure 16: Offline and Online Ablation Studies.

theoretically promises an $8\times$ reduction compared to BF16, the metadata overhead required for its fine-grained group quantization limits the maximum CR to approximately $5.33\times$. Consequently, KIVI achieves an average CR of only $4.40\times$. Similarly, DuoAttention (pruning-based) fails to achieve high compression without significant loss, as aggressively discarding tokens hurts long-context retrieval accuracy.

In contrast, our profiling engine successfully navigates the trade-off space, proving robust even on unseen data. *KVserve-Unified* serves as a highly effective default strategy when the workload type is unknown. By searching on a mixed dataset, it identifies a configuration that generalizes well, achieving an average CR of $7.42\times$ with a relative accuracy loss of less than 2%. Notably, on the unseen datasets, it maintains high fidelity without any task-specific tuning, demonstrating strong ability. When the workload type is known, *KVserve-Aware* unlocks superior performance by selecting specialized pipelines. It achieves an impressive average CR of $8.28\times$ —significantly outperforming all baselines—and peaks at $10.12\times$ on Multi-News. Furthermore, it maintains an average relative accuracy of 100.35% , exceeding the Default baseline. We attribute this capability to our *MixHQ* design, where the adaptive mixed-precision strategy selectively preserves significant features while filtering noise.

7.4 Ablation Studies

In this section, we conduct an ablation study to decouple and quantify the individual contributions of key algorithmic components within *KVserve*. We specifically examine the impact of optimization strategies on the efficiency of the Bayesian Profiling Engine and evaluate the necessity of the Service-Aware Online Controller for robust performance adaptation under dynamic serving conditions.

Efficiency of Offline Profiling Strategy. We evaluate the contribution of each optimization module within the Bayesian Profiling Engine by comparing the complete strategy (*KVserve*) against variants excluding Heterogeneous-Parameter Encoding (*w/o Enc*), Exploration-Exploitation Strategy (*w/o Exp*), Bi-Directional Pruning (*w/o Prune*), and Early-Stopping Mechanism (*w/o Stop*). As shown in Fig. 16 (Left), removing *Enc* and *Exp* leads to premature convergence trapped in local optima, yielding suboptimal CR of $8.96\times$ and $8.53\times$,

significantly lower than the global optimum of $9.31\times$. Conversely, ablating *Prune* and *Stop* allows finding the optimum but fails to converge within the allocated budget, exhausting the maximum 300 iterations. The full *KVserve* strategy synergizes these components, successfully identifying the global optimal configuration ($9.31\times$ CR) with superior sample efficiency, converging in just 194 iterations.

Robustness of Online Selection Policy. We assess the adaptability of the Service-Aware Online Controller under dynamic network conditions by monitoring end-to-end latency during bandwidth fluctuations (0–60s). The experiment compares the proposed residual-corrected approach (*KVserve*) against ablations lacking the Context Bandit (*w/o Bandit*) and the Online Controller (*w/o Controller*). As illustrated in Fig. 16 (Right), during significant bandwidth drops (shaded area, 20s–40s), the absence of the theoretical lower-envelope model (*w/o Controller*) results in severe latency spikes, peaking at nearly 0.9s, due to the selection of non-beneficial strategies. Furthermore, the lack of the online bandit (*w/o Bandit*) prevents the system from correcting runtime execution drift, leading to consistently higher latency compared to the full system. In contrast, *KVserve* achieves the lowest latency profile (stabilizing around 0.3s) by combining analytical modeling for baseline selection with bandit learning for real-time residual correction.

8 RELATED WORK

KV Cache Compression. Most KV cache compression methods center on quantization. Prior work improves the accuracy–compression tradeoff by (i) reshaping KV distributions before quantization to make them more amenable to low-bit representations [3, 29, 37, 44], (ii) allocating precision at finer granularity across layers/heads/tokens/channel to better match KV sensitivity [16, 26, 28], and (iii) reducing the runtime overhead of (de)compression through optimized implementations and kernels [19, 48]. We view these techniques as modular design knobs that can be instantiated as components and parameters in our strategy pool. In parallel, KV pruning reduces footprint by selectively retaining “important” states; it is largely orthogonal to quantization, but tends to incur larger quality loss at aggressive reduction levels [9, 18, 43]. In contrast to *KVserve*, most existing approaches are *service-agnostic*: they adopt fixed configurations and do not adapt to dynamic service context at runtime.

Disaggregated Serving Optimization. Recent serving systems increasingly optimize *disaggregated* inference. Phase-disaggregation systems redesign execution, scheduling, and to better utilize heterogeneous GPU pools, spanning PD separation, and scheduler-driven variants [10, 12, 15, 17, 33, 38, 49]. KV *state disaggregation* and KV-pool architectures optimize KV offloading, and reuse across requests, making

KV movement a first-class system concern [6, 21, 25, 35]. Elastic designs further generalize disaggregation by dynamically reallocating resources and parallelism as request mixes drift [7, 27, 42]. These system-level advances are complementary to our focus: we study *service-aware KV compression* as an orthogonal lever that can be embedded into both PD-separated and KV-disaggregated serving stacks.

9 CONCLUSION

Disaggregated LLM serving turns the KV cache from an internal GPU state into a massive, latency-critical payload, making KV movement a dominant bottleneck. KVServe re-thinks KV compression as a *service-state-dependent* decision problem rather than a fixed algorithm choice. By treating KV compression as a constrained, service-dependent control problem, KVServe enables robust end-to-end speedups across both PD separation and KV state disaggregation under dynamic workloads and bandwidth. Beyond KV caching, we believe the same principle applies to a broader class of networked state-movement workloads in modern disaggregated systems—e.g., parameter offloading, and embedding retrieval. Overall, KVServe establishes a service-aware foundation for disaggregated LLM serving, showing how KV movement can be optimized as a first-class, constraint-driven control problem. *This work does not raise any ethical issues.*

ACKNOWLEDGMENTS

This work was supported by the National Natural Science Foundation of China (Grant Nos. 62032023 and T2125013), the Innovation Funding of ICT, CAS (Grant No. E461050), and the National Key Research and Development Program of China (Grant No. 2025YFB3003702). The experiments were performed on the robotic AI-Scientist platform of Chinese Academy of Sciences.

REFERENCES

- [1] Amazon Web Services. 2026. Amazon EC2 FAQs. <https://aws.amazon.com/ec2/faqs/>. (2026). Accessed: 2026-01-29.
- [2] Muhammad Arslan, Hussam Ghanem, Saba Munawar, and Christophe Cruz. 2024. A Survey on RAG with LLMs. *Procedia computer science* 246 (2024), 3781–3790.
- [3] Saleh Ashkboos, Amirkeivan Mohtashami, Maximilian L Croci, Bo Li, Pashmina Cameron, Martin Jaggi, Dan Alistarh, Torsten Hoefer, and James Hensman. 2024. Quarot: Outlier-free 4-bit inference in rotated llms. *Advances in Neural Information Processing Systems* 37 (2024), 100213–100240.
- [4] Yushi Bai, Xin Lv, Jiajie Zhang, Hongchang Lyu, Jiankai Tang, Zhidian Huang, Zhengxiao Du, Xiao Liu, Aohan Zeng, Lei Hou, Yuxiao Dong, Jie Tang, and Juanzi Li. 2023. LongBench: A Bilingual, Multitask Benchmark for Long Context Understanding. (2023). [arXiv:cs.CL/2308.14508](https://arxiv.org/abs/2308.14508)
- [5] Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Ponde de Oliveira Pinto, Jared Kaplan, Harri Edwards, Yuri Burda, Nicholas Joseph, Greg Brockman, Alex Ray, Raul Puri, Gretchen Krueger, Michael Petrov, Heidy Khlaaf, Girish Sastry, Pamela Mishkin, Brooke Chan, Scott Gray, Nick Ryder, Mikhail Pavlov, Alethea Power, Lukasz Kaiser, Mohammad Bavarian, Clemens Winter, Philippe Tillet, Felipe Petroski Such, Dave Cummings, Matthias Plappert, Fotios Chantzis, Elizabeth Barnes, Ariel Herbert-Voss, William Hebgan Guss, Alex Nichol, Alex Paino, Nikolas Tezak, Jie Tang, Igor Babuschkin, Suchir Balaji, Shantanu Jain, William Saunders, Christopher Hesse, Andrew N. Carr, Jan Leike, Josh Achiam, Vedant Misra, Evan Morikawa, Alec Radford, Matthew Knight, Miles Brundage, Mira Murati, Katie Mayer, Peter Welinder, Bob McGrew, Dario Amodei, Sam McCandlish, Ilya Sutskever, and Wojciech Zaremba. 2021. Evaluating Large Language Models Trained on Code. (2021). [arXiv:cs.LG/2107.03374](https://arxiv.org/abs/2107.03374)
- [6] Weijian Chen, Shuibing He, Haoyang Qu, Ruidong Zhang, Siling Yang, Ping Chen, Yi Zheng, Baoxing Huai, and Gang Chen. 2025. IMPRESS: An Importance-Informed Multi-Tier Prefix KV Storage System for Large Language Model Inference. In *23rd USENIX Conference on File and Storage Technologies (FAST 25)*. 187–201.
- [7] Wenyan Chen, Chengzhi Lu, Huanle Xu, Kejiang Ye, and Chengzhong Xu. 2025. Multiplexing Dynamic Deep Learning Workloads with SLO-awareness in GPU Clusters. In *Proceedings of the Twentieth European Conference on Computer Systems*. 589–604.
- [8] Karl Cobbe, Vineet Kosaraju, Mohammad Bavarian, Mark Chen, Heewoo Jun, Lukasz Kaiser, Matthias Plappert, Jerry Tworek, Jacob Hilton, Reiichiro Nakano, Christopher Hesse, and John Schulman. 2021. Training Verifiers to Solve Math Word Problems. *arXiv preprint arXiv:2110.14168* (2021).
- [9] Alessio Devoto, Maximilian Jeblick, and Simon Jégou. 2025. Expected Attention: KV Cache Compression by Estimating Attention from Future Queries Distribution. *arXiv preprint arXiv:2510.00636* (2025). <https://arxiv.org/abs/2510.00636>
- [10] Jiangfei Duan, Runyu Lu, Haojie Duanmu, Xiuhong Li, Xingcheng Zhang, Dahua Lin, Ion Stoica, and Hao Zhang. 2024. Muxserve: Flexible spatial-temporal multiplexing for multiple llm serving. *arXiv preprint arXiv:2404.02015* (2024).
- [11] Haojie Duanmu, Zhihang Yuan, Xiuhong Li, Jiangfei Duan, Xingcheng Zhang, and Dahua Lin. 2024. Skvq: Sliding-window key and value cache quantization for large language models. *arXiv preprint arXiv:2405.06219* (2024).
- [12] Jingqi Feng, Yukai Huang, Rui Zhang, Sicheng Liang, Ming Yan, and Jie Wu. 2025. WindServe: Efficient Phase-Disaggregated LLM Serving with Stream-based Dynamic Scheduling. In *Proceedings of the 52nd Annual International Symposium on Computer Architecture*. 1283–1295.

- [13] Leo Gao, Jonathan Tow, Baber Abbasi, Stella Biderman, Sid Black, Anthony DiPofi, Charles Foster, Laurence Golding, Jeffrey Hsu, Alain Le Noac'h, Haonan Li, Kyle McDonell, Niklas Muennighoff, Chris Ociepa, Jason Phang, Laria Reynolds, Hailey Schoelkopf, Aviya Skowron, Lintang Sutawika, Eric Tang, Anish Thite, Ben Wang, Kevin Wang, and Andy Zou. 2024. The Language Model Evaluation Harness. (07 2024). <https://doi.org/10.5281/zenodo.12608602>
- [14] Aaron Grattafiori, Abhimanyu Dubey, Abhinav Jauhri, Abhinav Pandey, Abhishek Kadian, Ahmad Al-Dahle, Aiesha Letman, Akhil Mathur, Alan Schelten, Alex Vaughan, Amy Yang, Angela Fan, Anirudh Goyal, Anthony Hartshorn, Aobo Yang, Archi Mitra, Archie Sravankumar, Artem Korenev, Arthur Hinsvark, Arun Rao, Aston Zhang, Aurelien Rodriguez, Austen Gregerson, Ava Spataru, Baptiste Roziere, Bethany Biron, Binh Tang, Bobbie Chern, Charlotte Caucheteux, Chaya Nayak, Chloe Bi, Chris Marra, Chris McConnell, Christian Keller, Christophe Touret, Chunyang Wu, Corinne Wong, Cristian Canton Ferrer, Cyrus Nikolaidis, Damien Allonsius, Daniel Song, Danielle Pintz, Danny Livshits, Danny Wyatt, David Esiobu, Dhruv Choudhary, Dhruv Mahajan, Diego Garcia-Olano, Diego Perino, Dieuwke Hupkes, Egor Lakomkin, Ehab AlBadawy, Elina Lobanova, Emily Dinan, Eric Michael Smith, Filip Radenovic, Francisco Guzmán, Frank Zhang, Gabriel Synnaeve, Gabrielle Lee, Georgia Lewis Anderson, Govind Thattai, Graeme Nail, Gregoire Mialon, Guan Pang, Guillem Cucurell, Hailey Nguyen, Hannah Korevaar, Hu Xu, Hugo Touvron, Iliyan Zarov, Imanol Arrieta Ibarra, Isabel Kloumann, Ishan Misra, Ivan Evtimov, Jack Zhang, Jade Copet, Jaewon Lee, Jan Geffert, Jana Vranes, Jason Park, Jay Mahadeokar, Jeet Shah, Jelmer van der Linde, Jennifer Billock, Jenny Hong, Jenya Lee, Jeremy Fu, Jianfeng Chi, Jianyu Huang, Jiawen Liu, Jie Wang, Jiecao Yu, Joanna Bitton, Joe Spisak, Jongsoo Park, Joseph Rocca, Joshua Johnstun, Joshua Saxe, Junteng Jia, Kalyan Vasuden Alwala, Karthik Prasad, Kartikeya Upasani, Kate Plawiak, Ke Li, Kenneth Heafield, Kevin Stone, Khalid El-Arini, Krithika Iyer, Kshitiz Malik, Kuenley Chiu, Kunal Bhalla, Kushal Lakhotia, Lauren Rantala-Young, Laurens van der Maaten, Lawrence Chen, Liang Tan, Liz Jenkins, Louis Martin, Lovish Madaan, Lubo Malo, Lukas Blecher, Lukas Zandzaat, Luke de Oliveira, Madeline Muzzi, Mahesh Pasupuleti, Mannat Singh, Manohar Paluri, Marcin Kardas, Maria Tsimpoukelli, Mathew Oldham, Mathieu Rita, Maya Pavlova, Melanie Kambadur, Mike Lewis, Min Si, Mitesh Kumar Singh, Mona Hassan, Naman Goyal, Narjes Torabi, Nikolay Bashlykov, Nikolay Bogoychev, Niladri Chatterji, Ning Zhang, Olivier Duchenne, Onur Çelebi, Patrick Alrassy, Pengchuan Zhang, Pengwei Li, Petar Vasic, Peter Weng, Prajwal Bhargava, Pratik Dubal, Praveen Krishnan, Punit Singh Koura, Puxin Xu, Qing He, Qingxiao Dong, Ragavan Srinivasan, Raj Ganapathy, Ramon Calderer, Ricardo Silveira Cabral, Robert Stojnic, Roberta Raileanu, Rohan Maheswari, Rohit Girdhar, Rohit Patel, Romain Sauvestre, Ronnie Polidoro, Roshan Sumbaly, Ross Taylor, Ruan Silva, Rui Hou, Rui Wang, Saghar Hosseini, Sahana Chennabasappa, Sanjay Singh, Sean Bell, Seohyun Sonja Kim, Sergey Edunov, Shao-liang Nie, Sharan Narang, Sharath R-parthy, Sheng Shen, Shengye Wan, Shruti Bhosale, Shun Zhang, Simon Vandenhende, Soumya Batra, Spencer Whitman, Sten Sootla, Stephane Collet, Suchin Gururangan, Sydney Borodinsky, Tamar Herman, Tara Fowler, Tarek Sheasha, Thomas Georgiou, Thomas Scialom, Tobias Speckbacher, Todor Mihaylov, Tong Xiao, Ujjwal Karn, Vedanuj Goswami, Vibhor Gupta, Vignesh Ramanathan, Viktor Kerkez, Vincent Gonguet, Virginie Do, Vish Vogeti, Vitor Albiero, Vladan Petrovic, Weiwei Chu, Wenhan Xiong, Wenyan Fu, Whitney Meers, Xavier Martinet, Xiaodong Wang, Xiaofang Wang, Xiaoqing Ellen Tan, Xide Xia, Xinfeng Xie, Xu-chao Jia, Xuewei Wang, Yaelle Goldschlag, Yashesh Gaur, Yasmine Babaei, Yi Wen, Yiwen Song, Yuchen Zhang, Yue Li, Yuning Mao, Zacharie Delpierre Coudert, Zheng Yan, Zhengxing Chen, Zoe Papakipos, Aaditya Singh, Aayushi Srivastava, Abha Jain, Adam Kelsey, Adam Shajnfeld, Adithya Gangidi, Adolfo Victoria, Ahuva Goldstand, Ajay Menon, Ajay Sharma, Alex Boesenberg, Alexei Baevski, Allie Feinstein, Amanda Kallet, Amit Sangani, Amos Teo, Anam Yunus, Andrei Lupu, Andres Alvarado, Andrew Caples, Andrew Gu, Andrew Ho, Andrew Poulton, Andrew Ryan, Ankit Ramchandani, Annie Dong, Annie Franco, Anuj Goyal, Aparajita Saraf, Arkabandhu Chowdhury, Ashley Gabriel, Ashwin Bharambe, Assaf Eisenman, Azadeh Yazdan, Beau James, Ben Maurer, Benjamin Leonhardi, Bernie Huang, Beth Loyd, Beto De Paola, Bhargavi Paranjape, Bing Liu, Bo Wu, Boyu Ni, Braden Hancock, Bram Wasti, Brandon Spence, Brani Stojkovic, Brian Gamido, Britt Montalvo, Carl Parker, Carly Burton, Catalina Mejia, Ce Liu, Changhan Wang, Changkyu Kim, Chao Zhou, Chester Hu, Ching-Hsiang Chu, Chris Cai, Chris Tindal, Christoph Feichtenhofer, Cynthia Gao, Damon Civin, Dana Beaty, Daniel Kreymer, Daniel Li, David Adkins, David Xu, Davide Testuggine, Delia David, Devi Parikh, Diana Liskovich, Didem Foss, Dingkan Wang, Duc Le, Dustin Holland, Edward Dowling, Eissa Jamil, Elaine Montgomery, Eleonora Presani, Emily Hahn, Emily Wood, Eric-Tuan Le, Erik Brinkman, Esteban Arcaute, Evan Dunbar, Evan Smothers, Fei Sun, Felix Kreuk, Feng Tian, Filippos Kokkinos, Firat Ozgenel, Francesco Caggioni, Frank Kanayet, Frank Seide, Gabriela Medina Florez, Gabriella Schwarz, Gada Badeer, Georgia Swee, Gil Halpern, Grant Herman, Grigory Sizov, Guangyi Zhang, Guna Lakshminarayanan, Hakan Inan, Hamid Shojanazeri, Han Zou, Hannah Wang, Hanwen Zha, Haroun Habeeb, Harrison Rudolph, Helen Suk, Henry Aspegren, Hunter Goldman, Hongyuan Zhan, Ibrahim Damlaj, Igor Molybog, Igor Tufanov, Ilias Leontiadis, Irina-Elena Veliche, Itai Gat, Jake Weissman, James Geboski, James Kohli, Janice Lam, Japhet Asher, Jean-Baptiste Gaya, Jeff Marcus, Jeff Tang, Jennifer Chan, Jenny Zhen, Jeremy Reizenstein, Jeremy Teboul, Jessica Zhong, Jian Jin, Jingyi Yang, Joe Cummings, Jon Carvill, Jon Shepard, Jonathan McPhie, Jonathan Torres, Josh Ginsburg, Junjie Wang, Kai Wu, Kam Hou U, Karan Saxena, Kartikay Khandelwal, Katayoun Zand, Kathy Matosich, Kaushik Veeraraghavan, Kelly Michelena, Keqian Li, Kiran Jagadeesh, Kun Huang, Kunal Chawla, Kyle Huang, Lailin Chen, Lakshya Garg, Lavender A, Leandro Silva, Lee Bell, Lei Zhang, Liangpeng Guo, Licheng Yu, Liron Moshkovich, Luca Wehrstedt, Madian Khabsa, Manav Avalani, Manish Bhatt, Martynas Mankus, Matan Hasson, Matthew Lennie, Matthias Reso, Maxim Groshev, Maxim Naumov, Maya Lathi, Meghan Keneally, Miao Liu, Michael L. Seltzer, Michal Valko, Michelle Restrepo, Mihir Patel, Mik Vyatskov, Mikayel Samvelyan, Mike Clark, Mike Macey, Mike Wang, Miquel Jubert Hermoso, Mo Metanat, Mohammad Rastegari, Munish Bansal, Nandhini Santhanam, Natascha Parks, Natasha White, Navyata Bawa, Nayan Singhal, Nick Egebo, Nicolas Usunier, Nikhil Mehta, Nikolay Pavlovich Laptev, Ning Dong, Norman Cheng, Oleg Chernoguz, Olivia Hart, Omkar Salpekar, Ozlem Kalinli, Parkin Kent, Parth Parekh, Paul Saab, Pavan Balaji, Pedro Rittner, Philip Bontrager, Pierre Roux, Piotr Dollar, Polina Zvyagina, Prashant Ratanchandani, Pritish Yuvraj, Qian Liang, Rachad Alao, Rachel Rodriguez, Rafi Ayub, Raghotham Murthy, Raghu Nayani, Rahul Mitra, Rangaprabhu Parthasarathy, Raymond Li, Rebekkah Hogan, Robin Battey, Rocky Wang, Russ Howes, Rutu Rinott, Sachin Mehta, Sachin Siby, Sai Jayesh Bondu, Samyak Datta, Sara Chugh, Sara Hunt, Sargun Dhillon, Sasha Sidorov, Satadru Pan, Saurabh Mahajan, Saurabh Verma, Seiji Yamamoto, Sharadh Ramaswamy, Shaun Lindsay, Shaun Lindsay, Sheng Feng, Shenghao Lin, Shengxin Cindy Zha, Shishir Patil, Shiva Shankar, Shuqiang Zhang, Shuqiang Zhang, Sinong Wang, Sneha Agarwal, Soji Sajuyigbe, Soumith Chintala, Stephanie Max, Stephen Chen, Steve Kehoe, Steve Satterfield, Sudarshan Govindaprasad, Sumit Gupta, Summer Deng, Sungmin Cho, Sunny Virk, Suraj Subramanian, Sy Choudhury, Sydney Goldman, Tal Remez, Tamar Glaser, Tamara Best, Thilo Koehler, Thomas Robinson, Tianhe Li, Tianjun Zhang, Tim

- Matthews, Timothy Chou, Tzook Shaked, Varun Vontimitta, Victoria Ajayi, Victoria Montanez, Vijai Mohan, Vinay Satish Kumar, Vishal Mangla, Vlad Ionescu, Vlad Poenaru, Vlad Tiberiu Mihailescu, Vladimir Ivanov, Wei Li, Wenchen Wang, Wenwen Jiang, Wes Bouaziz, Will Constable, Xiaocheng Tang, Xiaojian Wu, Xiaolan Wang, Xilun Wu, Xinbo Gao, Yaniv Kleinman, Yanjun Chen, Ye Hu, Ye Jia, Ye Qi, Yenda Li, Yilin Zhang, Ying Zhang, Yossi Adi, Youngjin Nam, Yu, Wang, Yu Zhao, Yuchen Hao, Yundi Qian, Yunlu Li, Yuzi He, Zach Rait, Zachary DeVito, Zef Rosnbrick, Zhaoduo Wen, Zhenyu Yang, Zhiwei Zhao, and Zhiyu Ma. 2024. The llama 3 herd of models. *arXiv e-prints* (2024), arXiv-2407.
- [15] Ke Hong, Lufang Chen, Zhong Wang, Xiuhong Li, Qiuli Mao, Jianping Ma, Chao Xiong, Guanyu Wu, Buhe Han, Guohao Dai, Yun Liang, and Yu Wang. 2025. semi-PD: Towards Efficient LLM Serving via Phase-Wise Disaggregated Computation and Unified Storage. (2025). arXiv:cs.CL/2504.19867 <https://arxiv.org/abs/2504.19867>
- [16] Coleman Hooper, Sehoon Kim, Hiva Mohammadzadeh, Michael W Mahoney, Yakun S Shao, Kurt Keutzer, and Amir Gholami. 2024. Kvquant: Towards 10 million context length llm inference with kv cache quantization. *Advances in Neural Information Processing Systems* 37 (2024), 1270–1303.
- [17] Cunchen Hu, Heyang Huang, Liangliang Xu, Xusheng Chen, Chenxi Wang, Jiang Xu, Shuang Chen, Hao Feng, Sa Wang, Yungang Bao, Ninghui Sun, and Yizhou Shan. 2025. ShuffleInfer: Disaggregate LLM Inference for Mixed Downstream Workloads. *ACM Transactions on Architecture and Code Optimization* (2025).
- [18] Simon Jegou and Maximilian Jeblick. 2026. KVzap: Fast, Adaptive, and Faithful KV Cache Pruning. *arXiv preprint arXiv:2601.07891* (2026).
- [19] Bo Jiang, Taolue Yang, Youyuan Liu, Chengming Zhang, Xubin He, and Sian Jin. 2025. KVComp: A High-Performance, LLM-Aware, Lossy Compression Framework for KV Cache. *arXiv preprint arXiv:2509.00579* (2025).
- [20] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph Gonzalez, Hao Zhang, and Ion Stoica. 2023. Efficient memory management for large language model serving with pagedattention. In *Proceedings of the 29th symposium on operating systems principles*. 611–626.
- [21] Yuhang Li, Rong Gu, Chengying Huan, Zhibin Wang, Renjie Yao, Chen Tian, and Guihai Chen. 2025. Hotprefix: Hotness-aware kv cache scheduling for efficient prefix sharing in llm inference systems. *Proceedings of the ACM on Management of Data* 3, 4 (2025), 1–27.
- [22] Yuhong Li, Yingbing Huang, Bowen Yang, Bharat Venkitesh, Acyr Locatelli, Hanchen Ye, Tianle Cai, Patrick Lewis, and Deming Chen. 2024. Snapkv: Llm knows what you are looking for before generation. *Advances in Neural Information Processing Systems* 37 (2024), 22947–22970.
- [23] Yangning Li, Weizhi Zhang, Yuyao Yang, Wei-Chieh Huang, Yaozu Wu, Junyu Luo, Yuanchen Bei, Henry Peng Zou, Xiao Luo, Yusheng Zhao, Chunkit Chan, Yankai Chen, Zhongfen Deng, Yinghui Li, Hai-Tao Zheng, Dongyuan Li, Renhe Jiang, Ming Zhang, Yangqiu Song, and Philip S. Yu. 2025. Towards agentic rag with deep reasoning: A survey of rag-reasoning systems in llms. *arXiv preprint arXiv:2507.09477* 2 (2025).
- [24] Tengxuan Liu, Shiyao Li, Jiayi Yang, Tianchen Zhao, Feng Zhou, Xiaohui Song, Guohao Dai, Shengen Yan, Huazhong Yang, and Yu Wang. 2025. Pm-kvq: Progressive mixed-precision kv cache quantization for long-cot llms. *arXiv preprint arXiv:2505.18610* (2025).
- [25] Yuhang Liu, Yihua Cheng, Jiayi Yao, Yuwei An, Xiaokun Chen, Shaoting Feng, Yuyang Huang, Samuel Shen, Rui Zhang, Kuntai Du, and Junchen Jiang. 2025. Lmcache: An efficient KV cache layer for enterprise-scale LLM inference. *arXiv preprint arXiv:2510.09665* (2025).
- [26] Yuhang Liu, Hanchen Li, Yihua Cheng, Siddhant Ray, Yuyang Huang, Qizheng Zhang, Kuntai Du, Jiayi Yao, Shan Lu, Ganesh Ananthanarayanan, Michael Maire, Henry Hoffmann, Ari Holtzman, and Junchen Jiang. 2024. Cachegen: Kv cache compression and streaming for fast large language model serving. In *Proceedings of the ACM SIGCOMM 2024 Conference*. 38–56.
- [27] Zedong Liu, Shenggan Cheng, Guangming Tan, Yang You, and Dingwen Tao. 2025. Elasticmm: Efficient multimodal llms serving with elastic multimodal parallelism. *arXiv preprint arXiv:2507.10069* (2025).
- [28] Zirui Liu, Jiayi Yuan, Hongye Jin, Shaochen Zhong, Zhaozhuo Xu, Vladimir Braverman, Beidi Chen, and Xia Hu. 2024. Kivi: A tuning-free asymmetric 2bit quantization for kv cache. *arXiv preprint arXiv:2402.02750* (2024).
- [29] Yuxiao Ma, Huixia Li, Xiawu Zheng, Feng Ling, Xuefeng Xiao, Rui Wang, Shilei Wen, Fei Chao, and Rongrong Ji. 2024. Affinequant: Affine transformation quantization for large language models. *arXiv preprint arXiv:2403.12544* (2024).
- [30] NVIDIA. 2024. LLM Router NVIDIA. GitHub repository. (2024). <https://github.com/NVIDIA-AI-Blueprints/llm-router/tree/experimental>
- [31] NVIDIA. 2026. NVIDIA nvCOMP Developer. <https://developer.nvidia.com/nvcomp>. (2026).
- [32] Isaac Ong, Amjad Almahairi, Vincent Wu, Wei-Lin Chiang, Tianhao Wu, Joseph E Gonzalez, M Waleed Kadous, and Ion Stoica. 2024. Routellm: Learning to route llms with preference data. *arXiv preprint arXiv:2406.18665* (2024).
- [33] Pratyush Patel, Esha Choukse, Chaojie Zhang, Aashaka Shah, Íñigo Goiri, Saeed Maleki, and Ricardo Bianchini. 2024. Splitwise: Efficient generative llm inference using phase splitting. In *2024 ACM/IEEE 51st Annual International Symposium on Computer Architecture (ISCA)*. IEEE, 118–132.
- [34] Ruoyu Qin, Weiran He, Yaoyu Wang, Zheming Li, Xinran Xu, Yongwei Wu, Weimin Zheng, and Mingxing Zhang. 2026. Prefill-as-a-Service: KVCache of Next-Generation Models Could Go Cross-Datacenter. *arXiv preprint arXiv:2604.15039* (2026).
- [35] Ruoyu Qin, Zheming Li, Weiran He, Jialei Cui, Feng Ren, Mingxing Zhang, Yongwei Wu, Weimin Zheng, and Xinran Xu. 2025. Mooncake: Trading more storage for less computation—a {KVCache-centric} architecture for serving {LLM} chatbot. In *23rd USENIX conference on file and storage technologies (FAST 25)*. 155–170.
- [36] Philipp Schmid, Omar Sanseviero, Alvaro Bartolome, Leandro von Werra, Daniel Vila, Vaibhav Srivastav, Marc Sun, and Pedro Cuenca. 2024. Llama 3.1 – 405B, 70B & 8B with multilinguality and long context. <https://huggingface.co/blog/llama31>. (23 Jul 2024). Accessed: 2025-01-29.
- [37] Konrad Staniszewski and Adrian Łańcucki. 2025. KV Cache Transform Coding for Compact Storage in LLM Inference. (2025). arXiv:cs.CL/2511.01815 <https://arxiv.org/abs/2511.01815>
- [38] Biao Sun, Ziming Huang, Hanyu Zhao, Wencong Xiao, Xinyi Zhang, Yong Li, and Wei Lin. 2024. Llmunix: Dynamic scheduling for large language model serving. In *18th USENIX symposium on operating systems design and implementation (OSDI 24)*. 173–191.
- [39] Qian Tao, Wenyan Yu, and Jingren Zhou. 2025. Asymkv: Enabling 1-bit quantization of kv cache with layer-wise asymmetric quantization configurations. In *Proceedings of the 31st International Conference on Computational Linguistics*. 2316–2328.
- [40] Qwen Team. 2024. Qwen2.5: A Party of Foundation Models. (September 2024). <https://qwenlm.github.io/blog/qwen2.5/>
- [41] Chen Wang, Xunzhuo Liu, Yuhang Liu, Yue Zhu, Xiangxi Mo, Junchen Jiang, and Huamin Chen. 2025. When to Reason: Semantic Router for vLLM. *arXiv preprint arXiv:2510.08731* (2025).
- [42] Bingyang Wu, Shengyu Liu, Yinmin Zhong, Peng Sun, Xuanzhe Liu, and Xin Jin. 2024. Loongserve: Efficiently serving long-context large

- language models with elastic sequence parallelism. In *Proceedings of the ACM SIGOPS 30th Symposium on Operating Systems Principles*. 640–654.
- [43] Guangxuan Xiao, Jiaming Tang, Jingwei Zuo, Junxian Guo, Shang Yang, Haotian Tang, Yao Fu, and Song Han. 2024. Duoattention: Efficient long-context llm inference with retrieval and streaming heads. *arXiv preprint arXiv:2410.10819* (2024).
 - [44] Ceyu Xu, Yongji Wu, Xinyu Yang, Beidi Chen, Matthew Lentz, Danyang Zhuo, and Lisa Wu Wills. 2025. LLM. 265: Video Codecs are Secretly Tensor Codecs. In *Proceedings of the 58th IEEE/ACM International Symposium on Microarchitecture*. 445–460.
 - [45] An Yang, Baosong Yang, Binyuan Hui, Bo Zheng, Bowen Yu, Chang Zhou, Chengpeng Li, Chengyuan Li, Dayiheng Liu, Fei Huang, Guanting Dong, Haoran Wei, Huan Lin, Jialong Tang, Jialin Wang, Jian Yang, Jianhong Tu, Jianwei Zhang, Jianxin Ma, Jin Xu, Jingren Zhou, Jinze Bai, Jinzheng He, Junyang Lin, Kai Dang, Keming Lu, Keqin Chen, Kexin Yang, Mei Li, Mingfeng Xue, Na Ni, Pei Zhang, Peng Wang, Ru Peng, Rui Men, Ruize Gao, Runji Lin, Shijie Wang, Shuai Bai, Sinan Tan, Tianhang Zhu, Tianhao Li, Tianyu Liu, Wenbin Ge, Xiaodong Deng, Xiaohuan Zhou, Xingzhang Ren, Xinyu Zhang, Xipin Wei, Xuancheng Ren, Yang Fan, Yang Yao, Yichang Zhang, Yu Wan, Yunfei Chu, Yaqiong Liu, Zeyu Cui, Zhenru Zhang, and Zhihao Fan. 2024. Qwen2 Technical Report. *arXiv preprint arXiv:2407.10671* (2024).
 - [46] Cai Zefan, Yichi Zhang, Bofei Gao, Tianyu Liu, Keming Lu, Wayne Xiong, Yue Dong, Baobao Chang, Junjie Hu, and Wen Xiao. 2024. Pyramidkv: Dynamic kv cache compression based on pyramidal information funneling. *arXiv e-prints* (2024), arXiv–2406.
 - [47] Tianyi Zhang, Mohsen Hariri, Shaochen Zhong, Vipin Chaudhary, Yang Sui, Xia Hu, and Anshumali Shrivastava. [n. d.]. 70% Size, 100% Accuracy: Lossless LLM Compression for Efficient GPU Inference via Dynamic-Length Float (DFloat11). In *The Thirty-ninth Annual Conference on Neural Information Processing Systems*.
 - [48] Zeyu Zhang, Haiying Shen, Shay Vargaftik, Ran Ben Basat, Michael Mitzenmacher, and Minlan Yu. 2025. Hack: Homomorphic acceleration via compression of the key-value cache for disaggregated llm inference. In *Proceedings of the ACM SIGCOMM 2025 Conference*. 1245–1247.
 - [49] Yinmin Zhong, Shengyu Liu, Junda Chen, Jianbo Hu, Yibo Zhu, Xuanzhe Liu, Xin Jin, and Hao Zhang. 2024. DistServe: Disaggregating prefill and decoding for goodput-optimized large language model serving. In *18th USENIX Symposium on Operating Systems Design and Implementation (OSDI 24)*. 193–210.
 - [50] Zixuan Zhou, Xuefei Ning, Ke Hong, Tianyu Fu, Jiaming Xu, Shiyao Li, Yuming Lou, Luning Wang, Zhihang Yuan, Xiuhong Li, Shengen Yan, Guohao Dai, Xiao-Ping Zhang, Yuhang Dong, and Yu Wang. 2024. A survey on efficient inference for large language models. *arXiv preprint arXiv:2404.14294* (2024).